SEEING STARS - A CLASSIFICATION ADVENTURE

A PREPRINT

M.Sc. Bioinformatics
Aarhus University

M.Sc. Bioinformatics
Aarhus University

December 5, 2025

ABSTRACT

Stars and their constellations have always been part of human interest. But classifying constellations manually can be tedious and depends greatly on weather, visual conditions like light and level of knowledge. To ease and automate this task, several machine learning methods, as well as deep learning image classifier and object detector models have been developed. Since most of these approaches are relatively complex, this report aims to create a simple convolutional neural network for identifying star constellations. To achieve this, many experiments such as introducing a background class, applying data augmentations, dropout learning, weight-decay and early stopping are carried out, with the goal to improve the model's accuracy whilst handling problems like class imbalance and overfitting. As part of this project, six models were created, trained and evaluated. The final model performs with 97.26% test accuracy, indicating that the combination of techniques successfully improved model performance.

Keywords Star Constellation Recognition, Deep Learning, CNN, Astronomy

1 Introduction

1.1 Motivation

There has always been great interest in astronomy, from the complicated dynamics of the galaxy to simply identifying stars and their constellations in both a scientific, and also cultural way. Since finding constellations by eye alone can be hard and more and more night sky images are available, the demand for automating this task is rising. By today, star constellations in images can be detected using concepts like template matching, pattern recognition, sub-graph isomorphisms [1][2][3] or even deep learning models like YOLO (You-only-look-once), an object detection framework [3]. But all of these models use fairly advanced ideas for this rather simple problem. Therefore, this paper will investigate, how well a custom and 'simple' convolutional neural network (CNN) can learn to distinguish and classify star constellations.

1.2 Goals

Thus, the main focus is to train an image classifier on different star constellation images, evaluate its performance, and make it as precise as possible, using different fitting experiments and approaches that were taught during the lectures.

To be precise, we aim to:

1. Train a base model for image classification, keeping it as simple as possible.
2. Find and address arising problems through workflow iterations and adaptions such as class imbalance, overfitting, class separation, low accuracy, and creating a background class.
3. Create a final model with the highest possible prediction accuracy on the test set, that generalizes well and learns smoothly

In this context, it should be noted that the image classifier was created bearing the intention in mind, that it could be used in a sliding-window object detector as a next step. For that reason, a background class is introduced as class 17 in section 3.1.2.

In the following sections this report will first review related work that deals with the same dataset as the one used in this report, object detection, convolutional neural networks and experiments that have been done to improve the models. Then, the underlying methodology including the model architecture and experimental design will be presented, followed by the results. Finally, the report will discuss findings and problems that arised during the project realization.

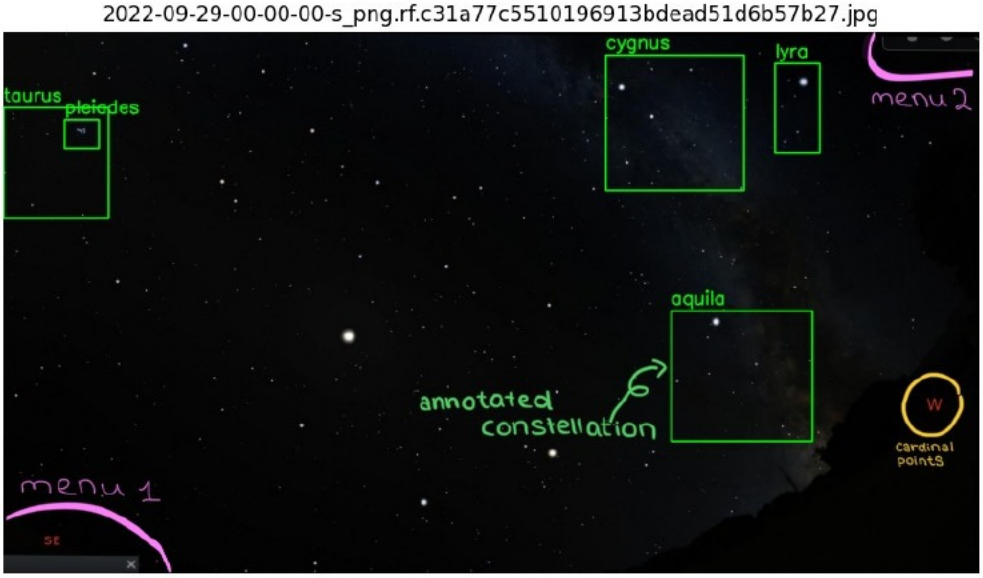


Figure 1: Example of an image from the Roboflow dataset [4]. It contains multiple annotated constellations that can overlap, but also cardinal points and two menus in the bottom left and top right corner.

1.3 Code Availability

All code, along with instructions for dataset acquisition, can be found on our GitHub repository [5]. Please don't hesitate to contact group1 on Slack if there are access problems.

2 Related work

2.1 Article 1 - Star and Constellation Recognition using YOLO framework

In this paper [3], YOLOv8 and YOLOv9 are used to train object detectors on the "Stars and Constellations Dataset" [4]. The YOLO framework, is different from ours, but it provides a good reference for evaluating our model. In this paper mean Average Precision at IoU = 0.50 (mAP50) of 95.7% is reached, with the YOLOv8 model outperforming v9, showing that task-specific models can outperform models with more parameters. In the article, another split of the dataset (80,10,10) than the one provided on the dataset website, is used, without providing the actual dataset splits, making exact comparison difficult. Still, it can be used as a benchmarking tool, for the sliding window object detector. This paper was mostly used for project inspiration, creating an outline and exploring the possibilities of the dataset.

2.2 Article 2 - Star-galaxy classification using deep convolutional neural networks

This study aims to use deep convolutional neural networks (ConvNets), to automate the process of training a star constellation classifier, and then compare the resulting model with the widely used machine learning algorithm *random forest* [6]. Kim et al. not only present a ConvNet that reaches the same performance as a random forest algorithm when classifying stars, but also give a broad introduction to CNN creation, deep learning parameters and common problems like overfitting, and possible solutions such as different data augmentations. The mentioned augmentations e.g. image rotation, reflection, translation and Gaussian noise were used as inspiration for handling upcoming problems during this

project, especially for handling overfitting.
Furthermore, it was surprising that Kim et al. used reflection as one of their augmentations, since that will never be observed on natural star constellation images.

2.3 Article 3 - Detection of Southern Hemisphere Constellations Using Convolutional Neural Networks

Riffo et al. use three different YOLO versions to detect 21 star constellations of the southern hemisphere [7]. Since the YOLO models are not used in our project, there was more focus on Riffo et al.'s motivation as well as their methods used for data augmentation and their training configuration. Regarding augmentations, rotation, translation, distortion and scaling are used, along with random variations in noise, brightness and contrast. The random variations idea will be one of the key ideas that will improve our model performance (see section 4 Results).

3 Methods

3.1 Data

This project uses the "Stars and Constellations Dataset" [4]. It contains 1750 images of the night sky, with bounding boxes and labels around the star constellations. In total, there are 16 different classes (see Figure 2). The data set was designed with the purpose of training and evaluating machine learning models for star constellation detection [3]. The data set is divided into training, validation, and test set, with 1530, 146, and 74 images, respectively. The same data split is used in this project. Since others have used it, it provides a good reference for evaluation.

Table 1: Annotation distribution across train, validation, and test splits, with mean bounding box sizes per class. Median values are used to determine background class size.

Class	Train	Valid	Test	Mean BBox Size (W × H)
aquila	282	15	18	207.66 × 212.06
bootes	284	20	17	221.09 × 274.32
canis_major	235	13	5	140.73 × 146.47
canis_minor	278	15	7	75.98 × 73.78
cassiopeia	455	25	30	182.59 × 186.05
cygnus	445	23	30	237.83 × 235.11
gemini	298	16	7	233.09 × 207.85
leo	251	14	14	330.91 × 203.25
lyra	449	25	25	108.60 × 100.61
moon	297	15	10	93.30 × 91.03
orion	272	13	10	133.08 × 201.66
pleiades	392	19	12	42.30 × 42.12
sagittarius	197	13	15	167.29 × 156.82
scorpius	213	16	14	240.92 × 208.33
taurus	306	11	10	173.35 × 162.63
ursa_major	488	23	29	267.89 × 298.95
Median annotations (background)	290	15	14	–
Total annotations	5142	276	253	178.52 × 177.53
Total classes	16	16	16	–

3.1.1 Preprocessing

To make a classifier for the 16 star constellations and a background class, the data went through different preprocessing steps. Annotations from each image were used to create cutouts of all of the constellations to use in the classifier. Making the cutouts, lead to the class distribution seen in Table 1. For normalization, the mean and standard deviation for each channel were calculated. These calculations were done on the full training dataset including the backgrounds. The cutouts were resized to 128x128 pixels. Choosing the right size is a trade-off between keeping as much information as possible for the model to train well, while keeping mind that it should be able to train relatively fast. There is a difference in the sizes of the constellations, and this size made sure that the smallest constellations were not too

pixelated. This could become an issue for the Pleiades, having a mean bounding box size of 42x42 (see Table 1). Most of the other constellations were shrunk down and reshaped by the preprocessing.

3.1.2 Creating background data

The dataset did not have a predefined background classes, and therefore it had to be constructed from the images. The background class should resemble the other classes in shape, size and quantity so that the only difference is the contents of the cutouts. The median number of cutouts in the other classes was used to determine the amount of cutouts in the background class. The annotated bounding boxes from the other classes were then used to make cutouts at random locations in the images. The background cutouts do not overlap with any of the constellations, but are allowed to overlap with other backgrounds. At first a maximum of 1 background cutout is sampled from each image. The difficulty of the background class is to make sure, that it represents the whole background of the images. Since the background covers the majority of each image, this is not a simple task.

Figure 2: All 16 star constellations including the moon which is not actually a constellation.

3.1.3 Issues with the data

On the data set website, there is no information about the creation of the images of the data set. Our best guess is, that that they are screenshots from a software similar to Stellarium, which is one of many planetarium software for looking at stars. During the making of this project, artifacts in many of the images that could be from a computer (see Figure 1), were observed. Potentially, this means that the model will not generalise as well, if using real pictures. It also means, that the model will have to learn these artifacts, as part of the background class. This problem could be solved by cropping the images, but that would be very time consuming, and beyond the scope of this project.
Some of the bounding boxes are wrongly annotated, highlighting a constellation that is not actually there. This could limit the models ability to train. In the classifier, the constellation cutouts are assumed to be 128x128 pixels, but the cutouts from the images come in all shapes and sizes, many of them being rectangular. This means that parts of the cutouts will be stretched before they are used in the model. This can cause some issues when implementing the object detector, depending on how the sliding window size and shape is chosen.
Additionally, there is some class imbalance, with the largest classes having twice as many cutouts as the smallest ones, in all three data splits (see Table 1). After running the base model it will be assessed , if this imbalance is enough to pose a problem or not. Also, the data set is relatively small with the largest classes containing around 500 images, and the smallest ones only around 200.
Another issue that has to be addressed, is the fact, that the sky can have many different orientations. Therefore, the model has to be robust to rotations of the star constellations.

3.2 Base model

3.2.1 Network architecture

For building the model, a relatively simple setup using Python 3.12.12 with the PyTorch module, was chosen. The model has 3 convolutional layers with 8, 16 and 32 filters respectively. Each of the convolutional layers use a padding of 1 and a filter size of 3x3, to keep the input and output size the same. Between each layer, max-pooling with a kernel size of 2 and stride of 2, is used. This means, that the input size is halved between each convolutional layer. In the

Table 2: Model architecture summary of the base model.

Layer (type)	Output Shape	Param #
Conv2d-1	[8, 128, 128]	224
MaxPool2d-2	[8, 64, 64]	0
Conv2d-3	[16, 64, 64]	1,168
MaxPool2d-4	[16, 32, 32]	0
Conv2d-5	[32, 32, 32]	4,640
MaxPool2d-6	[32, 16, 16]	0
Linear-7	[64]	524,352
Linear-8	[17]	1,105
Total params		531,489
Trainable params		531,489

end are two fully connected layers. The first fully connected layer reduces the feature representation to 64 dimensions, and the second fully connected layer maps this embedding to 17 output units, which correspond to the output class probabilities for classification. A summary of the model can be seen in Table 2, showing the shape and number of parameters at each layer. As the activation function, ReLU (Rectified Linear Unit) is used for simplicity, effectiveness and to avoid vanishing gradients.

3.2.2 Training and Training Configuration

For training the base model, a learning rate of 0.001 and 10 epochs were used, as well as Stochastic Gradient Descent (SGD) for optimization with a momentum of 0.9. Using momentum speeds up learning and smoothens it out, as each learning step considers 90% of the direction from the previous learning step. Additionally, a batch size of 32 was defined, to have a good representation of each class in every training batch. For the loss function, standard cross-entropy loss was used, as it was as well in a lot of the course labs. Nothing more complicated is necessary for this simple classification problem.

3.2.3 Diagnostics

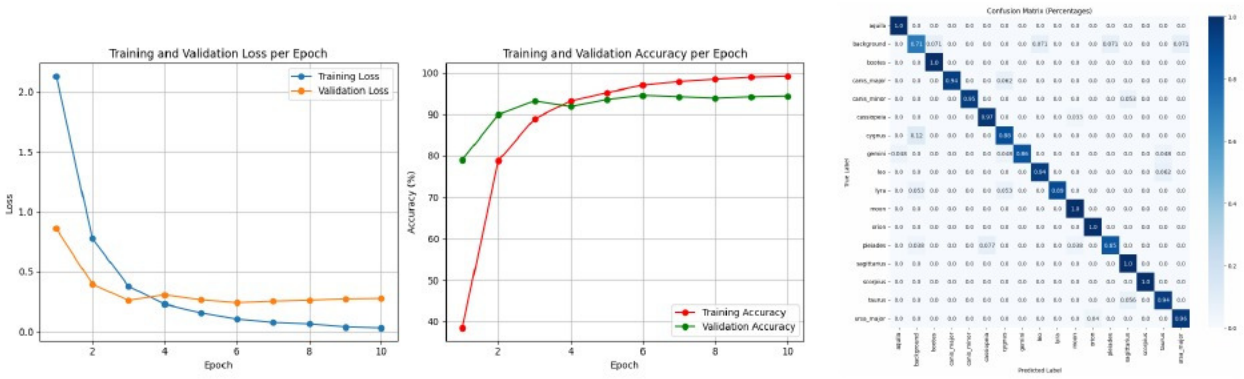
From Figure 3a it can be seen, that the base model is done training after around 6 epochs and the rest is overfitting. From epoch 4 the model begins to overfit since the training accuracy overtakes the validation accuracy. Another thing to note is, that the model actually performs quite well for a base model. It is to be expected, that star constellations are relatively easy to train on. The cutouts are almost black and white, and they have high contrast. Also, the cutouts for each constellation class should look more or less the same, except for rotation and cropping of the cutouts. This will be addressed through the experiments. Figure 3b shows that most classes are doing quite well, but still there is some work to do. The background class is especially bad, and some of the other classes can also be improved upon. The base model reaches an accuracy of 92.81% when evaluating on the test set.

3.3 Experiments and model versions

The base model introduces broad possibilities for improvement in many areas such as overfitting, different class prediction confidence and the overall accuracy, and the amount and variety of the training images. Therefore, the first experiment addresses the low prediction confidence of the background class, the second experiment will explore different data augmentations, experiment number three will deal with the problem of overfitting and experiment four tries to improve accuracy by applying weight-decay and early-stopping. The final experiment functions as a framework for fine tuning, testing interesting methods and evaluating, if they further improve accuracy.

3.3.1 V1 - More background images

Unlike a constellation, which will always contain more or less the same stars, the background class should contain everything that is not part of a constellation and therefore be very diverse. By scaling the initial number of background images by a factor of 2, 3, 5 and 10, the number of images, required for the model to make reliable predictions, were tested. The goal is, to find the factor that gives the best accuracy without creating a class-imbalance problem, reducing the models ability to classify the other classes.



(a) (b)

Figure 3: Overview of the base model evaluation figures. Loss and accuracy curves as well as a confusion matrix were used to identify problems for future workflow iterations. **(a)** Loss and accuracy curves of training and validation of the base model. The model overfits, struggling with generalization. Still, the validation accuracy is quite high. **(b)** Confusion matrix after training the base model. The overall percentages are very high and the model seems to be able to distinguish the classes well. The hardest class to predict is the background class with 71% confidence.

3.3.2 V2 - Data augmentation

There is a lot of improvement to be done in distinguishing the classes from each other. Also, the model needs to be robust to real-world challenges. Not every picture will look the same, since the constellations can be rotated, have different quality and placement within the image. Especially the quality can vary a lot in star pictures, since capturing stars is very hard and depends, not only on the tool used for capturing, but also on external noise like light pollution, bad weather conditions and star visibility. If the model is not trained on these special cases, it can confuse classes easier and mispredict class labels. Therefore, the following data augmentations will be implemented:

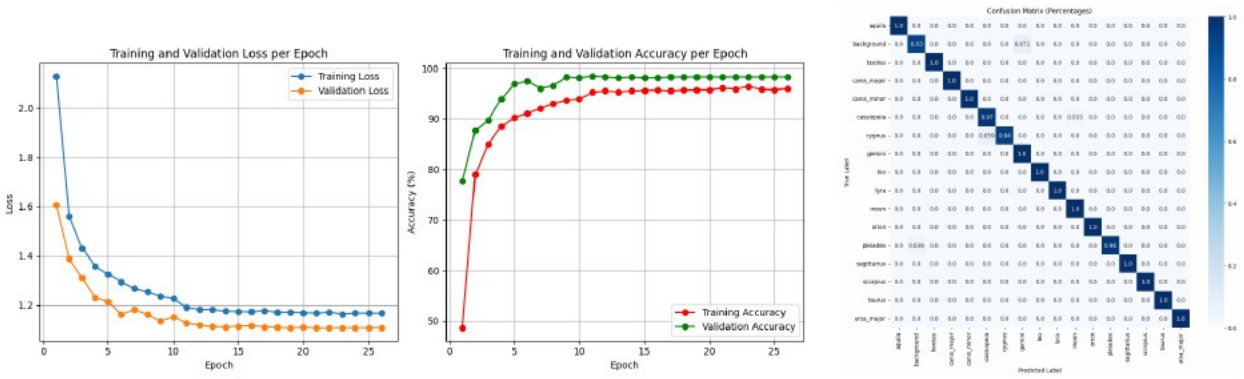
1. Image rotation - a constellation can be rotated depending on time and location of capturing.
2. Cropping - depending on an image, especially if it is an image containing multiple constellations, a constellation can only be visible partly, which will greatly influence classification.
3. Resizing (changing the aspect ratio of the images) - the model should be able to predict a constellation in every size.
4. Noise - as mentioned above, capturing star images is very hard and due to noise a picture can look very different. This augmentation will hopefully make the model more robust for this.
5. Shifting colour, brightness and contrast values - for the same reasons as mentioned above.

3.3.3 V3 - Better background images

So far, images have been chosen randomly, but this may not be the most effective way for training the model. In this experiment, hard negative mining is used to improve the quality of the background cutouts. 10000 new background cutouts were then created. The model was then used to predict on the background class alone and keep all of the wrong predictions. 450 of these backgrounds then replaced the old backgrounds and a second round of training of the model took place. The idea here is to train on the backgrounds that are most difficult for the model to distinguish. Decreasing the number of backgrounds while making them more qualified also has the potential to make the model better at predicting the other classes, since there is more balance between the classes.

3.3.4 V4 - Overfitting

After applying rotation, crop and random resized crop augmentations, the overfitting is almost gone completely. Nevertheless, the model should also be robust to noise and contrast variations, which negatively affect generalization and cause overfitting on the training data. This problem is tackled by first, using a dropout layer and second, reapply the noise, contrast and brightness augmentations as random variations as presented in Riffo et al. [7]. Through this, these heavy augmentations are only applied to 30% to 50% of the images.



(a) (b)

Figure 4: Overview of the V5 final model evaluation figures. **(a)** Loss and accuracy curves of training and validation of the V5 model. The model is not overfitting anymore and due to the learning rate scheduler, the curves are smooth. **(b)** Confusion matrix after training the V5 model. Classes are very well distinguishable, with over 90% confidence for each class. The background class still has the lowest confidence, but now it is at 93%.

3.3.5 V5 - Tuning hyperparameters and improving accuracy

The last performed experiment is to tune the hyperparameters such as learning rate, batch size and number of epochs, to obtain smoother learning curves and heighten the accuracy a bit more. To achieve this, various hyperparameter combinations are tested for finding the optimal configuration. To optimize learning and computation time in general during the hyperparameter search, weight-decay (regularization technique for penalizing large weights during training) and early-stopping are implemented. Additionally, experiments with the StepLR scheduler in PyTorch as a learning rate optimizer and label smoothing are conducted, to automate the hyperparameter settings. The learning rate and training time was also continuously monitored and updated throughout the experiments as needed.

4 Results

In the end, the V5 model reached an accuracy of 97.26% when predicting on the test set. 870 backgrounds were used for training, so a factor of three from the initial amount. This improved the model's ability to correctly classify the background class from 71% to 85% without making the model much worse in predicting the other classes and the accuracy went to 94.52% in total.

Including data augmentation improved the models' ability to predict on all classes. Rotations, random resizing and cropping were some of the important ones raising the accuracy to around 96.58%. At the same time, it almost eliminated overfitting completely. Furthermore, noise and colorJitter were added as augmentations on all of the inputs. This turned out to be to be too much for the model to handle, it failed to generalize and the overall accuracy decreased to 80%. To balance this, dropout learning and weight-decay were tried, which handled the overfitting but still kept the accuracy low. Inspired by Riffo et al. [7], we applied noise and colorJitter with a random variation component instead of applying it to all images. This made our model more robust for handling these artifacts, without making it overfit and decrease accuracy.
The rotation and resizing were from the beginning the most promising augmentations, because they capture known variability in the dataset. While noise and colouring can vary, it is probably not to the same extend, especially since the data is computer-generated.

Because the data augmentations already handled the issue of overfitting, further measures for this problem were not required and we focused on an even better class separation and improving of the accuracy. With implementing early-stopping we ensured not to overtrain our model. The final two experiments lead to the best performance of our model. Since finding a single learning rate that created a smooth curve was hard, a learning rate scheduler was used to slow down the learning rate every 10 epochs. This lead to the curves in Figure 4a. Finally, we attempted label smoothing for better class separation, but the accuracy stayed at 97.26%. Although looking at Figure 4b, it can be observed that the model's overall confidence when predicting is much higher in comparison to the base model (see Figure 3) and the model is very confident when separating the classes.

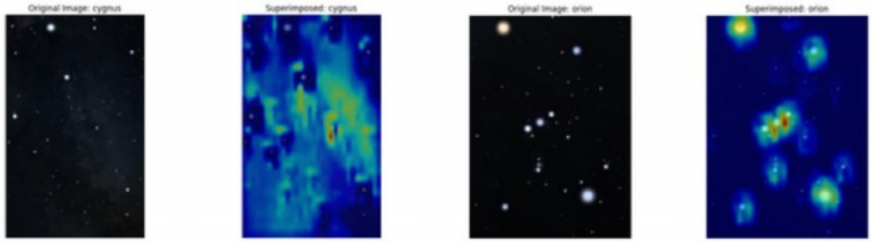


Figure 5: Heatmap to visualize how the model is interpreting regions in the input images. The left two images visualize the constellation *Cygnus* and the right two the constellation *Orion*. The model is focusing on the red and yellow areas when making predictions.

As a note to model 3, the experiment with hard negative mining did not provide any improvements to the model. It is possible that we could have made it work, but at this stage, with the other experiments, the model was already quite good and we did not pursue it further.

Figure 5 shows two constellations and a heatmap overlayed to show what the model is focusing on. This provides valuable insights to what the model has learned. On the right side is Orion, one of the most known and also most easy to predict classes for the model. The heatmap shows that the model actually looks for the stars like we would expect. On the left side is the star constellation Cygnus, which the model gets correct 94% of the times. Here, the heatmap is less clear. It looks at much more than just the stars. The background of the constellation is noisy. We suspect that the model has learned this noise instead of the stars which could explain why Cygnus is the star constellation it predicts with the lowest accuracy.

5 Discussion

This project has presented multiple deep learning models for the classification of star constellations, using different experiments and techniques. Through different methods such as data augmentation, multiplying and hard negative mining of the background images, dropout learning, weight-decay and early stopping we addressed problems of model generalization, overfitting, class-imbalance and prediction accuracy. Using plug-and-play tools such as YOLO might be easier and faster, but our results show, that a custom and simple CNN can indeed learn to distinguish and classify star constellations with good accuracy and generalization. The accuracy of our final model, predicting constellations on a test dataset, reaches 97.26%, which is much higher than expected.

Due to the dataset issues mentioned in section 3.1.3, the representativeness of the dataset cannot be guaranteed. Even though the model was improved a lot from the base model (see Figure 3) to the final model V5 (see Figure 4), it remains uncertain how much the model architecture accounts for this and how much is due to our dataset. While Chen et al. [3] claim that the dataset is real-world star images, it can be clearly seen in Figure 1 that the images are taken from a software. This makes images less noisy, more clear and also more similar to each other in one class, which is usually not given when dealing with star images. Furthermore, the dataset splits provided from the dataset give quite a small fraction of the images for validation and testing. This does make our evaluation less reliable. On the other hand, it gives the model more data to train on. It would have been good to have the same split as they do in the YOLOv8 paper for better comparison.

Due to the picture format, the background class might also be very specific for this dataset. Since the background is randomly sampled and can also contain a menu or a cardinal point cutout (see Figure 1), it might predict worse in terms of accuracy on images from other datasets.

As future work, this image classifier can be used in an object detector using a sliding-window approach. The authors recommend to train on more data or a different dataset to ensure good generalization and accuracy.

References

[1] Xiaoge Liu, Suyao Ji, and Jinzhi Wang. Constellation detection. 2015. URL https://api.semanticscholar.org/CorpusID:17361754.

[2] Arthur Dent. Star recognition using computer vision opencv. https://www.instructables.com/Star-Recognition-Using-Computer-Vision-OpenCV/. Accessed: December 4, 2025.

[3] Emmett Chen, Pallavi Bajpai, and Smriti H Bhandari. Star and constellation recognition using yolo framework. In *Proceedings of the 4th International Conference on AI-ML Systems*, AIMLSystems '24, New York, NY, USA, 2025. Association for Computing Machinery. ISBN 9798400711619. doi:10.1145/3703412.3704411. URL https://doi.org/10.1145/3703412.3704411.

[4] My Workspace. Constellation finder dataset. https://universe.roboflow.com/my-workspace-1ekf0/constellation-finder, sep 2022. URL https://universe.roboflow.com/my-workspace-1ekf0/constellation-finder. visited on 2025-11-10.

[5] . Seeing stars: A classification adventure. https://github.com/ /SeeingStarsAClassificationAdventure, 2025. Accessed: 2025-12-05.

[6] Edward J. Kim and Robert J. Brunner. Star–galaxy classification using deep convolutional neural networks. *Monthly Notices of the Royal Astronomical Society*, 464(4):4463–4475, 10 2016. ISSN 0035-8711. doi:10.1093/mnras/stw2672. URL https://doi.org/10.1093/mnras/stw2672.

[7] Vladimir Riffo, Sebastián Flores, Eduardo Chuy-Kan, and Víctor Ariza. Detection of southern hemisphere constellations using convolutional neural networks. *IEEE Access*, 13:96182–96189, 2025. doi:10.1109/ACCESS.2025.3575093.